

Kustomize 기반 개발/스테이징/운영 배포 가이드

1. 현재 문제점과 Kustomize 도입 목적

현재 방식의 문제

개발 yaml —(수동 복사/수정)—▶ 스테이징 yaml —(수동 복사/수정)—▶ 운영 yaml
↑ 개별 commit ↑ 개별 commit ↑ 개별 commit

- 환경마다 별도 YAML을 수동으로 복사·수정 후 commit
- 개발에서 검증된 설정이 운영에 그대로 반영되지 않음
- 이미지 태그, 리소스, 환경변수 등 환경별 차이점과 공통부가 혼재
- 휴먼에러 발생: replicas 누락, 이미지 태그 오기입, ConfigMap 불일치 등

Kustomize 도입 후

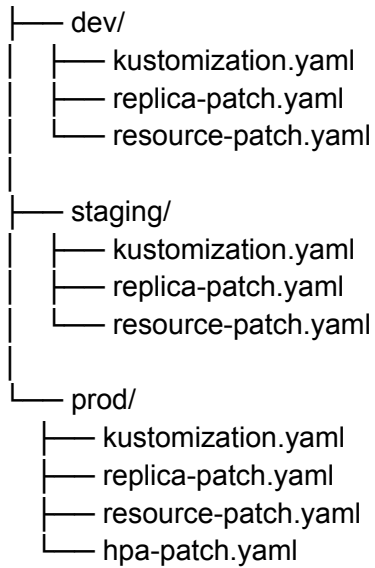
base (공통 yaml)

├─ overlays/dev ← 환경별 차이점만 패치
├─ overlays/staging ← base는 동일, 변경 사항만 오버레이
└─ overlays/prod ← base가 바뀌면 모든 환경에 자동 반영

- **base**를 한 번만 수정하면 전 환경에 반영
- 환경별 차이점만 **overlay**로 관리 → 변경 범위 최소화
- GitLab MR 리뷰 시 "무엇이 달라지는지" 명확하게 확인 가능

2. 디렉토리 구조

k8s/
├─ base/ # 공통 리소스 (모든 환경의 기준)
│ ├── kustomization.yaml
│ ├── deployment.yaml
│ ├── service.yaml
│ ├── configmap.yaml
│ └── hpa.yaml
└─ overlays/ # 환경별 오버레이



3. Base 리소스 작성

base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
# 공통 라벨 (모든 리소스에 자동 추가)
```

```
commonLabels:
```

```
  app: my-app
```

```
  managed-by: kustomize
```

```
resources:
```

```
  - deployment.yaml
```

```
  - service.yaml
```

```
  - configmap.yaml
```

base/deployment.yaml

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: my-app
```

```
spec:
```

```
  replicas: 1
```

```
  # 기본값 (overlay에서 환경별로 변경)
```

```

selector:
  matchLabels:
    app: my-app
template:
  metadata:
    labels:
      app: my-app
  spec:
    containers:
      - name: my-app
        image: registry.example.com/my-app:latest # overlay에서 태그 변경
        ports:
          - containerPort: 8080
        envFrom:
          - configMapRef:
              name: my-app-config
        resources: # 기본 리소스 (overlay에서 환경별 조정)
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 512Mi
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 10
          periodSeconds: 5
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10

```

base/service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  type: ClusterIP

```

```
ports:
  - port: 80
    targetPort: 8080
    protocol: TCP
selector:
  app: my-app
```

base/configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-app-config
data:
  LOG_LEVEL: "info"
  APP_PORT: "8080"
```

4. 환경별 Overlay 작성

4.1 개발 환경 (dev)

overlays/dev/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
# base 참조
```

```
resources:
```

```
- ../../base
```

```
# 네임스페이스 설정
```

```
namespace: dev
```

```
# 리소스명에 prefix 추가 (선택사항)
```

```
namePrefix: dev-
```

```
# 이미지 태그 변경 ← 환경별 이미지 관리의 핵심
```

```
images:
```

```
- name: registry.example.com/my-app
```

```
  newTag: dev-abc1234      # 개발 브랜치의 최신 커밋 해시
```

```
# 환경별 패치 적용
```

patches:

- path: replica-patch.yaml
- path: resource-patch.yaml

ConfigMap 값 오버라이드

configMapGenerator:

- name: my-app-config
 - behavior: merge # base의 ConfigMap에 병합
 - literals:
 - LOG_LEVEL=debug # 개발 환경은 debug 로깅
 - SPRING_PROFILES_ACTIVE=dev

overlays/dev/replica-patch.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-app

spec:

replicas: 1 # 개발은 1개면 충분

overlays/dev/resource-patch.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-app

spec:

template:

spec:

containers:

- name: my-app

resources:

requests:

cpu: 100m

memory: 128Mi

limits:

cpu: 500m

memory: 512Mi

4.2 스테이징 환경 (staging)

overlays/staging/kustomization.yaml

apiVersion: kustomize.config.k8s.io/v1beta1

kind: Kustomization

resources:

- ../../base

namespace: staging

namePrefix: stg-

images:

- name: registry.example.com/my-app
newTag: v1.2.3-rc1 # RC(Release Candidate) 태그

patches:

- path: replica-patch.yaml
- path: resource-patch.yaml

configMapGenerator:

- name: my-app-config

behavior: merge

literals:

- LOG_LEVEL=info
- SPRING_PROFILES_ACTIVE=staging

overlays/staging/replica-patch.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-app

spec:

replicas: 2 # 운영과 유사한 구성으로 검증

overlays/staging/resource-patch.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-app

spec:

template:

spec:

containers:

```
- name: my-app
  resources:
    requests:
      cpu: 250m
      memory: 256Mi
    limits:
      cpu: "1"
      memory: 1Gi
```

4.3 운영 환경 (prod)

overlays/prod/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
resources:
```

- ../../base
- hpa.yaml # 운영 전용 HPA 추가

```
namespace: production
namePrefix: prod-
```

```
images:
```

- name: registry.example.com/my-app
- newTag: v1.2.3 # 정식 릴리스 태그

```
patches:
```

- path: replica-patch.yaml
- path: resource-patch.yaml
- path: hpa-patch.yaml

```
configMapGenerator:
```

- name: my-app-config
- behavior: merge
- literals:
 - LOG_LEVEL=warn
 - SPRING_PROFILES_ACTIVE=prod

overlays/prod/replica-patch.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: my-app
spec:
  replicas: 3          # 고가용성
```

overlays/prod/resource-patch.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
        - name: my-app
          resources:
            requests:
              cpu: 500m
              memory: 512Mi
            limits:
              cpu: "2"
              memory: 2Gi
```

overlays/prod/hpa.yaml (운영 전용 리소스)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

5. 환경별 차이점 요약

항목	dev	staging	prod
namespace	dev	staging	production
replicas	1	2	3 (HPA 3~10)
image tag	dev-<commit>	v1.2.3-rc1	v1.2.3
CPU request	100m	250m	500m
Memory request	128Mi	256Mi	512Mi
LOG_LEVEL	debug	info	warn
HPA	없음	없음	있음

6. 빌드 및 배포 명령어

빌드 결과 미리보기 (dry-run)

각 환경의 최종 YAML 확인

```
kubectl kustomize overlays/dev
```

```
kubectl kustomize overlays/staging
```

```
kubectl kustomize overlays/prod
```

파일로 출력하여 diff 비교

```
kubectl kustomize overlays/dev > /tmp/dev.yaml
```

```
kubectl kustomize overlays/staging > /tmp/staging.yaml
```

```
kubectl kustomize overlays/prod > /tmp/prod.yaml
```

```
diff /tmp/staging.yaml /tmp/prod.yaml
```

직접 적용

개발 환경 배포

```
kubectl apply -k overlays/dev
```

스테이징 환경 배포

```
kubectl apply -k overlays/staging
```

```
# 운영 환경 배포
```

```
kubectl apply -k overlays/prod
```

변경사항 확인 (적용 전)

```
# 현재 클러스터 상태와 비교
```

```
kubectl diff -k overlays/prod
```

7. GitLab CI/CD 파이프라인 연동

.gitlab-ci.yml

```
stages:
```

- build
- deploy-dev
- deploy-staging
- deploy-prod

```
variables:
```

```
  REGISTRY: registry.example.com/my-app
```

```
# _____
```

```
# 이미지 빌드
```

```
# _____
```

```
build:
```

```
  stage: build
```

```
  script:
```

```
    - docker build -t ${REGISTRY}:${CI_COMMIT_SHORT_SHA} .
```

```
    - docker push ${REGISTRY}:${CI_COMMIT_SHORT_SHA}
```

```
    # 태그 푸시 시 릴리스 태그도 생성
```

```
    - |
```

```
      if [ -n "$CI_COMMIT_TAG" ]; then
```

```
        docker tag ${REGISTRY}:${CI_COMMIT_SHORT_SHA}
```

```
        ${REGISTRY}:${CI_COMMIT_TAG}
```

```
        docker push ${REGISTRY}:${CI_COMMIT_TAG}
```

```
      fi
```

```
# _____
```

```
# 개발 환경 배포 (develop 브랜치 push 시 자동)
```

```
# -----
deploy-dev:
  stage: deploy-dev
  script:
    # kustomize로 이미지 태그만 변경 → base는 건드리지 않음
    - cd k8s/overlays/dev
    - kustomize edit set image ${REGISTRY}:dev-${CI_COMMIT_SHORT_SHA}
    - kubectl apply -k .
  environment:
    name: development
  only:
    - develop
```

```
# -----
# 스테이징 배포 (release 브랜치 push 시 자동)
# -----
deploy-staging:
  stage: deploy-staging
  script:
    - cd k8s/overlays/staging
    - kustomize edit set image ${REGISTRY}:${CI_COMMIT_SHORT_SHA}
    - kubectl apply -k .
  environment:
    name: staging
  only:
    - /^releaseV.*$/
```

```
# -----
# 운영 배포 (태그 push 시, 수동 승인 필요)
# -----
deploy-prod:
  stage: deploy-prod
  script:
    - cd k8s/overlays/prod
    - kustomize edit set image ${REGISTRY}:${CI_COMMIT_TAG}
    - kubectl apply -k .
  environment:
    name: production
  when: manual          # 수동 승인 후 배포
  only:
    - tags
```

8. 환경 승격(Promotion) 워크플로우

Kustomize 도입의 핵심 가치는 환경 간 승격 시 **base**를 재사용하는 것입니다.

승격 흐름

1. 개발자가 **base/** 또는 **overlay** 수정 → MR 생성
2. **develop** 머지 → **dev** 자동 배포
3. **dev**에서 검증 완료
4. **release** 브랜치 생성 → **staging** 자동 배포
5. **staging**에서 QA 완료
6. 태그 생성 (v1.2.3) → **prod** 수동 승인 후 배포

핵심 원칙

1) **base** 변경 = 모든 환경에 영향

base/deployment.yaml에서 readinessProbe 수정 시
dev, staging, prod 모든 환경에 동일하게 반영됨
→ 더 이상 "개발에선 됐는데 운영에선 안 됨" 발생하지 않음

2) 환경별 차이는 **overlay**에서만

운영에만 tolerations 추가가 필요하면
overlays/prod/에 패치 파일만 추가
→ base를 건드리지 않으므로 dev/staging에 영향 없음

3) 이미지 태그만 바꿔서 승격

staging에서 검증된 이미지를 prod에 그대로 사용
overlays/prod/kustomization.yaml의 images.newTag만 변경

9. 자주 발생하는 휴먼에러와 Kustomize 해결책

에러 1: 이미지 태그 오기입

AS-IS: 개발자가 운영 yaml에 이미지 태그를 직접 타이핑 → 오타 발생
TO-BE: CI/CD에서 kustomize edit set image 명령으로 자동 설정

에러 2: ConfigMap 불일치

AS-IS: 개발에 추가한 환경변수를 운영에 반영 안 함

TO-BE: base/configmap.yaml에 키 추가 → 전 환경에 자동 반영
환경별 값만 overlay의 configMapGenerator에서 오버라이드

에러 3: replicas/리소스 설정 실수

AS-IS: 운영에 개발 설정(replicas: 1)을 실수로 배포

TO-BE: 환경별 replica-patch.yaml로 분리 → 실수 여지 제거
base 변경이 replicas에 영향을 주지 않음

에러 4: 새 리소스 누락

AS-IS: 개발에 추가한 Service/Ingress를 운영 yaml에 안 넣음

TO-BE: base/에 리소스 추가 → 모든 환경에 자동 포함

10. 유용한 팁

환경별 빌드 결과 비교

dev와 prod의 차이점만 빠르게 확인

```
diff <(kubectl kustomize overlays/dev) <(kubectl kustomize overlays/prod)
```

kustomize edit 명령어 활용

이미지 태그 변경 (kustomization.yaml 자동 수정)

```
cd overlays/prod
```

```
kustomize edit set image registry.example.com/my-app:v1.2.4
```

네임스페이스 변경

```
kustomize edit set namespace production
```

리소스 추가

```
kustomize edit add resource ingress.yaml
```

Strategic Merge Patch vs JSON Patch

Strategic Merge Patch (기본, 위 예시 방식)

→ 특정 필드만 덮어쓰기, 간단한 변경에 적합

JSON Patch (복잡한 변경에 사용)

patches:

- target:
 - kind: Deployment
 - name: my-app
- patch: |
 - op: add
 - path: /spec/template/spec/containers/0/env/-
 - value:
 - name: NEW_VAR
 - value: "hello"

Secrets 관리 (secretGenerator)

overlays/prod/kustomization.yaml

secretGenerator:

- name: my-app-secrets
- literals:
 - DB_PASSWORD=changeme # 실제로는 CI/CD 변수 또는 외부 시크릿 관리 도구 사용
- options:
 - disableNameSuffixHash: true # 이름에 해시 미추가

주의: 시크릿은 GitLab CI/CD Variables, HashiCorp Vault, AWS Secrets Manager 등 외부 도구와 연동하는 것을 권장합니다. 평문을 Git에 commit하지 마세요.

11. 마이그레이션 체크리스트

기존 개별 YAML 방식에서 Kustomize로 전환할 때 확인할 항목입니다.

- [] 모든 환경의 기존 YAML을 비교하여 공통부를 **base**로 추출
- [] 환경별 차이점(replicas, 리소스, 환경변수, 이미지 태그)을 **overlay** 패치로 분리
- [] `kubectl kustomize` 결과와 기존 **YAML**이 동일한지 **diff**로 검증
- [] 개발 환경에서 먼저 **Kustomize** 적용 후 정상 동작 확인
- [] GitLab CI/CD 파이프라인에 `kustomize edit set image` 연동
- [] 기존 개별 YAML 파일 아카이브 또는 삭제
- [] 팀원 대상 **Kustomize** 사용법 공유